Iteration G3: Testing Our JavaScript Functionality

Now that we have application-level functionality in JavaScript code, we're going to need to have tests in place to ensure that the function not only works as intended but continues to work as we make changes to the application.

Testing this functionality involves a lot of steps: visiting the store, selecting an item, adding that item to the cart, clicking checkout, filling in a few fields, and selecting a payment type. And from a testing perspective, we're going to need both a Rails server and a browser.

To accomplish this, Rails makes use of the popular Google Chrome web browser and Capybara,[60] which is a tool that drives this automation. Microsoft Edge and Mozilla's Firefox are also supported, as is Apple's Safari once Allow Remote Automation is enabled via the Develop menu.

Tests that pull together a complete and integrated version of the software are called *system tests*, and that's exactly what we'll be doing: we'll be testing a full end-to-end scenario with a web browser, web server, our application, and a database.

When we created scaffolds in previous chapters, Rails created system tests for us that performed basic checks. Let's run those now to make sure they're passing and that system testing is working. If you're using a virtual machine, you'll need to make one change before running the tests. Edit `test/application_system_test_case.rb` and change `:chrome` to `:headless_chrome` so that the system tests use a browser that doesn't need to pop up on the screen, like so:

rails7/depot_p/test/application_system_test_case.rb

```
require "test_helper"

class ApplicationSystemTestCase < ActionDispatch::SystemTestCase
» driven_by :selenium, using: :headless_chrome, screen_size: [1400, 1400]
end
```